

Day 03/5 — Create a Simple REST API with Controller Basics

Today you move from **Spring Beans and Dependency Injection** to actually creating **HTTP REST endpoints**.

The main idea is:

```
Client
  ↓ HTTP Request
Controller
  ↓
Java Code
  ↓
HTTP Response
```

1. What is a REST API?

A **REST API** allows clients such as browsers, Postman, mobile apps, or React applications to communicate with a backend through HTTP.

For example:

```
GET /hello
```

The server might return:

```
Hello from Spring Boot
```

Or:

```
GET /books
```

might return:

```
[
  {
    "id": 1,
    "title": "Java Basics"
  },
  {
    "id": 2,
    "title": "Spring Boot"
  }
]
```

2. What is a Controller?

A **Controller** is the part of your Spring application that receives HTTP requests and sends HTTP responses.

For example:

```
Browser/Postman
  ↓
GET /hello
  ↓
GreetingController
  ↓
"Hello"
  ↓
HTTP Response
```

Interview definition

A controller handles incoming HTTP requests, invokes the appropriate application logic, and returns an HTTP response to the client.

3. @RestController

@RestController tells Spring:

This class contains REST endpoints and the return values of its methods should normally be written directly into the HTTP response body.

Example:

```
@RestController
public class GreetingController {

    @GetMapping("/hello")
    public String hello() {
        return "Hello from Spring Boot";
    }
}
```

Now:

```
GET /hello
```

returns:

```
Hello from Spring Boot
```

4. @RestController vs @Controller

You will encounter both.

@Controller

Traditionally used with Spring MVC views.

```
@Controller
public class HomeController {
}
```

@RestController

Designed for REST APIs.

```
@RestController
public class BookController {
}
```

For the REST APIs you're building:

Use @RestController.

5. @RequestMapping

@RequestMapping can define a common URL prefix for an entire controller.

Example:

```
@RestController
@RequestMapping("/api")
public class BookController {

    @GetMapping("/books")
    public String books() {
        return "Books";
    }
}
```

The actual endpoint becomes:

```
GET /api/books
```

Because:

```
@RequestMapping("/api")
+
@GetMapping("/books")
=
/api/books
```

6. @GetMapping

@GetMapping maps an HTTP **GET request** to a Java method.

```
@GetMapping("/hello")
public String hello() {
    return "Hello";
}
```

When the client sends:

```
GET /hello
```

Spring executes:

```
hello()
```

Common HTTP methods

HTTP Method	Common purpose
GET	Read data
POST	Create data
PUT	Update/replace data

PATCH	Partially update data
DELETE	Delete data

For today's lesson, focus on **GET**.

7. Activity 1 – GET /hello

Create:

```
@RestController
public class GreetingController {

    @GetMapping("/hello")
    public String hello() {
        return "Hello from Spring Boot";
    }
}
```

Start your application.

Then open:

```
http://localhost:8080/hello
```

You should see:

```
Hello from Spring Boot
```

The complete flow:

```
Browser
  ↓
GET /hello
  ↓
Spring
```

```
↓  
GreetingController  
↓  
hello()  
↓  
"Hello from Spring Boot"  
↓  
Browser
```

8. Returning Plain Text

A controller method can return a simple String.

```
@GetMapping("/hello")  
public String hello() {  
    return "Hello";  
}
```

The response body is:

```
Hello
```

This is a good first REST endpoint because you don't have to worry about JSON yet.

9. @PathVariable

A **path variable** is a value contained inside the URL path.

For example:

```
/hello/Muneeb
```

Here:

Muneeb

is part of the URL path.

You can capture it using `@PathVariable`.

```
@GetMapping("/hello/{name}")
public String hello(@PathVariable String name) {
    return "Hello " + name;
}
```

Now:

GET /hello/Muneeb

returns:

Hello Muneeb

And:

GET /hello/Ali

returns:

Hello Ali

10. How {name} Works

Look at:


```
@GetMapping("/hello/{name}")
```

The {name} is a placeholder.

Then:

```
@PathVariable String name
```

tells Spring:

Take the value from {name} in the URL and put it into this Java variable.

For:

```
/hello/Muneeb
```

Spring effectively gives:

```
name = "Muneeb";
```

Then:

```
return "Hello " + name;
```

produces:

```
Hello Muneeb
```

11. Path Variable with Explicit Name

You can also write:

```
@GetMapping("/hello/{name}")
public String hello(@PathVariable("name") String personName) {
    return "Hello " + personName;
}
```

Here:

```
URL variable = name
Java variable = personName
```

This is useful when the names are different.

12. @RequestParam

A request parameter is provided after ? in the URL.

Example:

```
/hello?name=Muneeb
```

Here:

```
name=Muneeb
```

is a query parameter.

Java:

```
@GetMapping("/hello")
public String hello(@RequestParam String name) {
    return "Hello " + name;
}
```

Request:

```
GET /hello?name=Muneeb
```

Response:

```
Hello Muneeb
```

13. Path Variable vs Request Parameter

This is one of the most important concepts today.

Path Variable

```
/hello/Muneeb
```

Java:

```
@PathVariable String name
```

URL structure:

```
/hello/{name}
```

Request Parameter

```
/hello?name=Muneeb
```

Java:

```
@RequestParam String name
```

URL structure:

```
/hello?name=value
```

Easy way to remember

PathVariable = value is part of the path.

RequestParam = value comes after ?.

14. Side-by-Side Example

Path variable

```
@GetMapping("/users/{id}")
public String getUser(@PathVariable int id) {
    return "User ID: " + id;
}
```

Request:

```
GET /users/10
```

Response:

```
User ID: 10
```

Request parameter

```
@GetMapping("/users")
public String getUser(@RequestParam int id) {
```

```
    return "User ID: " + id;
}
```

Request:

```
GET /users?id=10
```

Response:

```
User ID: 10
```

Both can carry values, but their URL structures are different.

15. Optional Request Parameter

You can make a request parameter optional.

```
@GetMapping("/hello")
public String hello(
    @RequestParam(defaultValue = "Guest") String name) {

    return "Hello " + name;
}
```

Now:

```
/hello?name=Muneeb
```

returns:

```
Hello Muneeb
```

But:

```
/hello
```

returns:

```
Hello Guest
```

16. Returning JSON

REST APIs commonly return JSON.

Suppose we have:

```
public record Book(  
    int id,  
    String title  
) {  
}
```

Then:

```
@GetMapping("/book")  
public Book book() {  
    return new Book(1, "Java Basics");  
}
```

Spring converts the Java object into JSON.

Response:

```
{  
  "id": 1,
```

```
"title": "Java Basics"
}
```

You didn't manually write:

```
return "{\"id\":1,\"title\":\"Java Basics\"}";
```

Spring handles the conversion.

17. How Java Object Becomes JSON

At a beginner level, understand it like this:

```
Java Object
  ↓
Spring/Jackson
  ↓
JSON
  ↓
HTTP Response
```

For example:

```
new Book(1, "Java Basics")
```

becomes:

```
{
  "id": 1,
  "title": "Java Basics"
}
```

Spring Boot commonly uses **Jackson** for JSON serialization/deserialization.

Interview point

Spring Boot uses an HTTP message converter, commonly backed by Jackson, to convert Java objects into JSON responses.

You don't need to manually perform the conversion for normal REST endpoints.

18. Java Record for JSON

A record is very convenient for simple response data.

```
public record Book(  
    int id,  
    String title  
) {  
}
```

Then:

```
@GetMapping("/book")  
public Book getBook() {  
    return new Book(1, "Java Basics");  
}
```

Response:

```
{  
  "id": 1,  
  "title": "Java Basics"  
}
```

19. Returning Multiple Books

Use a List.

```
@GetMapping("/books")
public List getBooks() {
    return List.of(
        new Book(1, "Java Basics"),
        new Book(2, "Spring Boot")
    );
}
```

Response:

```
[
  {
    "id": 1,
    "title": "Java Basics"
  },
  {
    "id": 2,
    "title": "Spring Boot"
  }
]
```

20. Activity 3 — GET /books

Complete controller:

```
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.RestController;

import java.util.List;

@RestController
public class BookController {

    @GetMapping("/books")
    public List getBooks() {
        return List.of(
```

```
        new Book(1, "Java Basics"),  
        new Book(2, "Spring Boot")  
    );  
}  
}
```

Record:

```
public record Book(  
    int id,  
    String title  
) {  
}
```

Now:

```
GET /books
```

returns:

```
[  
  {  
    "id": 1,  
    "title": "Java Basics"  
  },  
  {  
    "id": 2,  
    "title": "Spring Boot"  
  }  
]
```

21. Complete Day-3 Example

You can combine today's activities into one application.

GreetingController

```

package com.example.demo;

import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.PathVariable;
import org.springframework.web.bind.annotation.RequestParam;
import org.springframework.web.bind.annotation.RestController;

@RestController
public class GreetingController {

    @GetMapping("/hello")
    public String hello() {
        return "Hello from Spring Boot";
    }

    @GetMapping("/hello/{name}")
    public String helloName(@PathVariable String name) {
        return "Hello " + name;
    }

    @GetMapping("/hello-query")
    public String helloQuery(@RequestParam String name) {
        return "Hello " + name;
    }
}

```

Book record

```

package com.example.demo;

public record Book(
    int id,
    String title
) {
}

```

BookController

```

package com.example.demo;

```

```
import org.springframework.web.bind.annotation.GetMapping;
import org.springframework.web.bind.annotation.RestController;

import java.util.List;

@RestController
public class BookController {

    @GetMapping("/books")
    public List getBooks() {
        return List.of(
            new Book(1, "Java Basics"),
            new Book(2, "Spring Boot")
        );
    }
}
```

22. Test All Endpoints

Assuming your application runs on:

```
localhost:8080
```

Endpoint 1

```
GET http://localhost:8080/hello
```

Response:

```
Hello from Spring Boot
```

Endpoint 2

```
GET http://localhost:8080/hello/Muneeb
```

Response:

```
Hello Muneeb
```

Endpoint 3

```
GET http://localhost:8080/hello-query?name=Muneeb
```

Response:

```
Hello Muneeb
```

Endpoint 4

```
GET http://localhost:8080/books
```

Response:

```
[
  {
    "id": 1,
    "title": "Java Basics"
  },
  {
    "id": 2,
    "title": "Spring Boot"
  }
]
```

23. Browser vs Postman

Browser

A browser is perfect for simple GET requests.

Enter:

```
http://localhost:8080/hello
```

or:

```
http://localhost:8080/books
```

You can immediately see the response.

Postman

Postman is more useful when developing APIs because you can explicitly select:

```
GET  
POST  
PUT  
DELETE
```

and later work with:

- headers
- request body
- JSON
- authentication
- query parameters
- path parameters
- status codes

For today's GET endpoints, **both browser and Postman are sufficient.**

24. @RequestMapping with Controllers

Instead of:

```
@RestController
public class BookController {

    @GetMapping("/books")
    public List getBooks() {
        ...
    }
}
```

you can organize your API:

```
@RestController
@RequestMapping("/api/books")
public class BookController {

    @GetMapping
    public List getBooks() {
        ...
    }
}
```

Now the endpoint is:

```
GET /api/books
```

The class-level mapping provides the common prefix.

This becomes especially useful as your API grows.

25. Keep Controller Methods Small

Avoid putting business logic directly inside controllers.

Not ideal

```
@GetMapping("/books")
public List getBooks() {
```

```
// lots of business logic

// database logic

// validation logic

// calculations

// more logic

return books;
}
```

Instead, later you'll use:

```
Controller
  ↓
Service
  ↓
Repository
  ↓
Database
```

For example:

```
@RestController
public class BookController {

    private final BookService bookService;

    public BookController(BookService bookService) {
        this.bookService = bookService;
    }

    @GetMapping("/books")
    public List getBooks() {
        return bookService.getBooks();
    }
}
```


The controller's job is primarily to handle the HTTP layer.

26. Controller + Service Architecture

Connect Day 2 and Day 3:

```
HTTP Request
  ↓
BookController
  ↓
BookService
  ↓
BookRepository
  ↓
Database
```

For today's hardcoded example:

```
HTTP Request
  ↓
BookController
  ↓
Hardcoded List
  ↓
JSON Response
```

Later, the hardcoded list will be replaced by database access.

27. Common Mistake — Path Variable vs Request Parameter

Incorrect thinking

Seeing:

```
/hello/Muneeb
```

and using:

```
@RequestParam
```

That's wrong because Muneeb is part of the URL path.

Correct:

```
@GetMapping("/hello/{name}")  
public String hello(@PathVariable String name) {  
    return "Hello " + name;  
}
```

For:

```
/hello?name=Muneeb
```

use:

```
@GetMapping("/hello")  
public String hello(@RequestParam String name) {  
    return "Hello " + name;  
}
```

28. Common Mistake — Forgetting {}

Wrong:

```
@GetMapping("/hello/name")  
public String hello(@PathVariable String name) {
```

```
    return "Hello " + name;
}
```

Correct:

```
@GetMapping("/hello/{name}")
public String hello(@PathVariable String name) {
    return "Hello " + name;
}
```

{name} tells Spring that name is a dynamic path value.

29. Common Mistake — Wrong Endpoint

If you have:

```
@GetMapping("/books")
```

you must request:

```
GET /books
```

not:

```
GET /book
```

and not:

```
POST /books
```

The HTTP method and URL both matter.

30. Common Mistake — Returning JSON Manually

Don't normally do this:

```
@GetMapping("/book")
public String book() {
    return "{\"id\":1,\"title\":\"Java Basics\"}";
}
```

Instead:

```
@GetMapping("/book")
public Book book() {
    return new Book(1, "Java Basics");
}
```

Let Spring/Jackson handle JSON conversion.

31. Interview Questions

Q1. What is a controller?

A controller handles incoming HTTP requests and returns appropriate HTTP responses.

Q2. What does @RestController do?

It marks a class as a REST controller. Its methods can handle HTTP requests, and their return values are generally written directly to the response body.

Q3. What is @GetMapping?

| @GetMapping maps an HTTP GET request to a controller method.

Q4. What is @RequestMapping?

| @RequestMapping maps HTTP requests to controllers or methods and can be used at class level to define a common URL prefix.

Q5. What is @PathVariable?

| It extracts a value from a variable part of the URL path.

Example:

```
/users/10
```

```
@PathVariable int id
```

Q6. What is @RequestParam?

| It extracts a query parameter from the URL.

Example:

```
/users?id=10
```

```
@RequestParam int id
```

Q7. Difference between @PathVariable and @RequestParam?

| @PathVariable extracts values from the URL path, while @RequestParam extracts values from query parameters.

Q8. How does Spring return a Java object as JSON?

Spring's HTTP message conversion mechanism, commonly using Jackson, serializes the Java object into JSON and writes it to the HTTP response body.

Q9. Should business logic be placed inside a controller?

Generally no. Controllers should remain thin and delegate business logic to service classes.

Q10. What happens when you return a List from a REST controller?

Spring serializes the list and its objects into a JSON array.

Java:

```
return List.of(
    new Book(1, "Java"),
    new Book(2, "Spring")
);
```

JSON:

```
[
  {
    "id": 1,
    "title": "Java"
  },
  {
    "id": 2,
    "title": "Spring"
  }
]
```

32. Important Interview Scenario

Interviewer:

What happens when a client sends GET /hello/Muneeb?

A strong beginner answer:

Spring's request mapping matches /hello/{name} with the incoming URL. Spring extracts Muneeb as the path variable and passes it to the controller method through @PathVariable. The method generates the greeting, and Spring sends the returned value as the HTTP response body.

33. Day 3 Mental Model

Memorize this flow:

```
Client
  ↓
HTTP Request
  ↓
@RestController
  ↓
@GetMapping
  ↓
Controller Method
  ↓
Java Object / String
  ↓
Spring HTTP Message Converter
  ↓
JSON / Text
  ↓
HTTP Response
  ↓
Client
```

And these two URL patterns:

Path Variable:

/hello/Muneeb

↑

@PathVariable

Request Parameter:

/hello?name=Muneeb

↑

@RequestParam

Day-3 must remember

1. @RestController → REST controller.
2. @RequestMapping → common/request mapping.
3. @GetMapping → GET endpoint.
4. @PathVariable → value inside URL path.
5. @RequestParam → value after ?.
6. Java objects can be automatically converted to JSON.
7. List becomes a JSON array.
8. Keep controllers **small/thin**.
9. Business logic belongs in the **service layer**.
10. Test your endpoints using a **browser or Postman**.